

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконавля:

студентка групи НАІ-243

Задорожна Анна Леонідівна

Перевірив:

ас. Баранов О.М.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт `/api/generate`. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	251	4
Час, c total_duration	25,76	11,89
Токенів/с	13,34	26,57

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5/ 5	2 / 5
eval_count	1293	193,6
Час, c	321,75	10,38

Типова помилка інструктивної моделі: відповідь «20 грн», отримана через неправильне трактування умови: модель помилково визначила різницю між ручкою та зошитом як 10 грн замість правильних 100 грн. Також у іншому запуску модель отримала «50 грн» та «55 грн», неправильно склавши рівняння.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальній моделі?

Відповідь: Додаткові токени міркувальна модель витратила на внутрішній аналіз задачі та перевірку можливих варіантів відповіді. Це збільшує eval_count і час генерації.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: Найбільшу перевагу вони мають у задачах на математику, логіку, програмування та послідовне міркування. Там важливо не просто запам'ятати відповідь, а правильно пройти кілька кроків розв'язання.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Міркувальні моделі додатково навчають виконувати складні задачі через послідовне міркування та перевірку результату. Інструктивні моделі більше орієнтовані на виконання заданих користувачем інструкцій і формування корисної відповіді.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Ні, більша кількість токенів не означає автоматично кращу модель. Вона може витратити більше токенів на міркування, але все одно помилятися.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: Швидкість залежить від архітектури моделі, її розміру, кількості токенів, що генеруються, та можливостей комп'ютера. У твоєму експерименті qwen3:4b генерував значно більше токенів міркування, тому його відповіді були набагато довшими за часом.

Висновок за завданням 1

Експеримент показав, що два показники – швидкість генерації та загальний час відповіді – не можна плутати. Перший характеризує обладнання й розмір моделі, другий – обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, витрати відрізняються у 2,2 разів. На задачі про ручку й зошит ситуація протилежна – саме розписаний хід дозволив уникнути інтуїтивно привабливої, але неправильної відповіді. Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами різниця складе 83,0 с проти 2574,0 с, що робить міркувальну модель непридатною для налагодження.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Виконано запити до двох моделей – llama3.2:3b (інструктивна) та qwen3:4b (міркувальна) – через OpenAI-сумісний ендпойнт локального сервісу Ollama. Для кожної моделі зроблено по два запуски з різними system-повідомленнями: «Ти лаконічний технічний асистент» та «Відповідай лише українською, максимум двома реченнями». Користувачий запит однаковий за для всіх запусків (варіант 2): «Поясни різницю між стеком і чергою». Для кожного запуску збережено текст відповіді, довжину в символах, кількість речень, мову та дані prompt_tokens, completion_tokens, total_tokens.

Таблиця 3: Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	1333	українська, але з помилками/змішаними словами	11	ні
llama3.2:3b	українською, 2 речення	261	українська, але з помилками	2	так
qwen3:4b	лаконічний асистент	266	українська	5	Так
qwen3:4b	українською, 2 речення	179	українська	2	Так

Таблиця 4: Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	51	392	443
llama3.2:3b	українською, 2 речення	55	89	143

qwen3:4b	лаконічний асистент	53	1828	1881
qwen3:4b	українською, 2 речення	62	977	1039

Типова помилка інструктивної моделі llama3.2:3b: при system «лаконічний асистент» вона згенерувала 1333 символи замість короткої відповіді, вживаючи вигадані слова. При system «максимум двома реченнями» вона формально виконала обмеження (2 речення, 261 символ), але повністю втратила зміст: описала стек як «конструкцію для захисту від вітру та вологи». Це демонструє, що мала інструктивна модель здатна дотримати форму, але не сенс.

Міркувальна модель qwen3:4b в обох випадках дала коректні відповіді: 265 символів при «лаконічному» system і 179 символів (рівно 2 речення) при «2 речення». Проте її completion_tokens виявилися у 5–20 разів більшими за llama3.2:3b (1828 і 977 проти 392 і 89) через прихований блок thinking, який не видно користувачу, але який входить у витрати токенів.

Відповіді на питання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: qwen3:4b – виконала і формально (2 речення), і за змістом. llama3.2:3b – формально 2 речення, але зміст повністю втрачено.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Відповідь: Ні, обидві моделі відповіли українською. Але llama3.2:3b вживала вигадані слова, що свідчить про слабке знання мови.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Відповідь: prompt_tokens зросла на 4-9 токенів, бо нове system-повідомлення довше. Різниця мала, бо обидва повідомлення короткі.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Відповідь: Бо qwen3:4b – міркувальна модель: вона генерує прихований thinking, який входить у completion_tokens. llama3.2:3b – інструктивна, генерує лише видиму відповідь.

Відповіді на питання (крок 7)

1. Я порівнював інструктивну модель (llama3.2:3b) та міркувальну (qwen3:4b) з різними system-повідомленнями. Чому моделі порізноmu дотримуються прямих інструкцій щодо довжини та мови відповіді?

Відповідь: Бо вони навчені на різних даних і з різними методами (SFT, RLHF). qwen3:4b краще слідує інструкціям завдяки якіснішому тренуванню на instruction-following. llama3.2:3b – менша модель, гірше тримає обмеження.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі – чи справді воно має більший пріоритет?

Відповідь: Формально обидва – просто токени в контексті. Але модель навчена, що system задає «рамку» поведінки, а user – конкретний запит. Тому system зазвичай має вищий пріоритет, але не абсолютний.

3. Чому міркувальна модель витрачає більше вихідних tokenів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Відповідь: Бо completion_tokens включає і thinking, і response. qwen3:4b генерує 1000–1800 tokenів роздумів перед короткою відповіддю. llama3.2:3b не має цього блоку.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, – яку вона виконає?

Відповідь: Зазвичай перемагає system, бо він задає рамку. Але якщо user-інструкція дуже конкретна й наполеглива, модель може її виконати. Поведінка залежить від моделі та тренування.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Відповідь: Менша модель має менше параметрів і гірше узагальнює. Вона частіше «змішує» мови, вигадує слова або ігнорує обмеження. Великі моделі краще тримають інструкції.

Висновок за завданням 2

Роль system-повідомлення. System-повідомлення – найдешевший і найшвидший спосіб керувати поведінкою моделі: не потрібне ні донавчання, ні зміна коду, достатньо змінити один рядок. Проте його надійність не абсолютна й залежить від моделі. qwen3:4b виконала обидві інструкції точно – і щодо довжини, і щодо мови, і щодо змісту. Llama3.2:3b виконала інструкцію «2 речення» лише формально, а «лаконічність» проігнорувала повністю (1333 символи замість короткої відповіді). Отже, сама лише інструкція не гарантує потрібного формату – малі моделі гірше тримають обмеження.

Що робити, якщо модель ігнорує інструкцію. По-перше, зробити інструкцію конкретнішою: не «будь лаконічним», а «відповідай рівно 2 речення, не більше 200 символів». По-друге, продублювати ключову вимогу в user-повідомленні – це підвищує шанс виконання. По-третє, використати більшу або міркувальну модель, яка краще слідує інструкціям. По-четверте, перевіряти done_reason і довжину відповіді програмно, а не покладатися лише на «слухняність» моделі.

Ціна керування. У qwen3:4b навіть коротка видима відповідь коштує 977–1828 completion_tokens через thinking. Тому керування поведінкою через system-повідомлення в міркувальних моделях дешеве за кодом, але дороге за токенами. Для простих задач, де потрібна лише форма, інструктивна модель може бути вигіднішою; для задач, де важливі і форма, і зміст, – міркувальна.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Виконано запити до двох моделей – Llama3.2:3b (локальна) та gemini-3.1-flash-lite (хмарна, від Google AI Studio) – через OpenAI-сумісний ендпойнт. Для обох моделей використано однаковий код (providers.py + compare.py), який

відрізняється лише вмістом словника PROVIDERS: для локальної – `base_url = http://localhost:11434/v1`, `api_key = ollama`; для хмарної – `base_url = https://generativelanguage.googleapis.com/v1beta/openai/`, `api_key` з файлу `.env`. Користувачський запит однаковий для всіх запусків (варіант 2): «Поясни різницю між стеком і чергою. Коротко». Для кожного запуску збережено текст відповіді, довжину в символах, час відповіді та дані `prompt_tokens`, `completion_tokens`, `total_tokens`.

Таблиця 5: Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.1-flash-lite
Час відповіді, с	47.60	2.83
completion_tokens	101	303
Довжина відповіді, символів	282	898
Мова відповіді	українська, з помилками	українська
Фактичних помилок у відповіді	100 % (полная маячня)	0

Відповіді на питання (крок 5)

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: Фактично жодного рядка логіки – змінилися лише значення в словнику PROVIDERS (`base_url`, `api_key`, `model`). Функція `make_client()` і цикл у `compare.py` залишилися ідентичними. Це демонструє, що формат OpenAI став універсальним стандартом.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: Хмарна (`gemini-3.1-flash-lite`) – повністю правильна: пояснила LIFO/FIFO, навела аналогії (стопка тарілок, черга в магазині) і приклади

використання. Локальна (llama3.2:3b) – повністю хибна: описала стек як «пристрійку до будинку», а чергу – як «вежову з віконцями».

3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

Відповідь: Так, і те, і інше. Фактично – 100% помилок (відповідь не стосується питання). З мовою – грубі помилки й вигадані слова.

4. Хмарна модель відповіла швидше за локальну – за рахунок чого, якщо запит летить через інтернет?

Відповідь: За рахунок потужного обладнання Google (спеціалізовані TPU/GPU), яке обробляє мільярди параметрів за мілісекунди. Локальна модель рахує на CPU твого ноутбука, тому 47.60 с проти 2.83 с – різниця в 16.8 разів. Затримка мережі (~100–200 мс) мізерна порівняно з різницею в обчислювальній потужності.

Відповіді на питання (крок 6)

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: Бо OpenAI був першим, хто запропонував зручний і простий API для LLM. Коли з'явилися інші провайдери, їм було вигідно підтримувати той самий формат – щоб розробники могли легко перемикатися. Це знижує витрати на інтеграцію й дозволяє використовувати вже написаний код.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: Розмір – один з ключових факторів, але не єдиний. Важливі також якість тренувальних даних, методи навчання (SFT, RLHF), токенизація та архітектура. У нашому експерименті llama3.2:3b (3B) дала повну маячню, а gemini-3.1-flash-lite (ймовірно, значно більша) – точну відповідь. Але навіть мала модель може дати непогану відповідь, якщо вона добре натренована.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь: Обладнанням. Google використовує спеціалізовані TPU/GPU, які обробляють мільярди параметрів за мілісекунди. Локальний ноутбук рахує на CPU – це в десятки разів повільніше. Затримка мережі (100–200 мс) – мізерна порівняно з різницею в обчислювальній потужності.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: Усі дані, які ти надсилаєш, потрапляють на чужий сервер – їх можуть логувати, аналізувати або використовувати для тренування. У комерційних проєктах: підписують NDA з провайдером, використовують enterprise-тарифи з гарантіями, або розгортають локальні моделі для чутливих даних.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь: Rate limit – це обмеження на кількість запитів за одиницю часу (наприклад, 15 запитів/хвилину). Розробники обходять це: чергують кілька ключів; додають паузи між запитами; використовують кілька провайдерів; переходять на платний тариф; кешують відповіді.

Висновок за завданням 3

Формат OpenAI – універсальний стандарт: код пишеться один раз, а перемикання між локальною й хмарною моделлю зводиться до зміни трьох значень у словнику. У нашому експерименті хмарна gemini-3.1-flash-lite виявилася у 16.8 разів швидшою (2.83 с проти 47.60 с), дала у 3 рази більше токенів і повністю правильну відповідь (0 помилок). Локальна llama3.2:3b дала 100% помилок – повну маячню про «пристрої до будинку», з грубими мовними помилками.

Коли обирати локальну: коли дані конфіденційні, немає інтернету, або потрібна повна автономність. Коли хмарну: коли потрібна якість, швидкість і точність, а дані не чутливі. У реальних проєктах часто використовують гібридний підхід: локальна модель для простих/чутливих задач, хмарна – для складних.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Виконано вимірювання швидкості генерації для локальної (llama3.2:3b) та хмарної (gemini-3.1-flash-lite) моделей через OpenAI-сумісний ендпойнт. Для вимірювання швидкості використано benchmark.py, який через stream=True фіксує TTFT (час до першого токена), загальний час і швидкість генерації (символів/с). Для кожної моделі (llama3.2:3b – локальна, gemini-3.1-flash-lite – хмарна) зроблено прогрів, а потім по 3 повтори для трьох промптів (варіант 2): короткий – «Столиця Японії?», середній – «Поясни в одному абзаці, що таке рекурсія.», довгий – «Напиши інструкцію з 10 пунктів, як налаштувати робоче середовище розробника.». Для кожного промпту взято медіану з трьох запусків. Окремо виміряно холодний (після ollama stop llama3.2:3b) і теплий старт локальної моделі.

Таблиця 6: Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2.13	2.77	11.2
локальна	llama3.2:3b	середній	2.139	6.373	50.1
локальна	llama3.2:3b	довгий	2.177	37.665	63.1
хмарна	gemini-3.1-flash-lite	короткий	1.366	1.369	20.5
хмарна	gemini-3.1-flash-lite	середній	1.386	2.025	283.0
хмарна	gemini-3.1-flash-lite	довгий	1.046	5.152	678.7

Таблиця 7: Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	2.122	2.13
Всього, с	2.696	2.975

Для наочного порівняння швидкості генерації побудовано стовпчикову діаграму (див. рис. 1), яка показує швидкість у символах за секунду для всіх шести

комбінацій «модель × промпт». Локальна модель llama3.2:3b позначена зеленим кольором, хмарна gemini-3.1-flash-lite – рожевим.

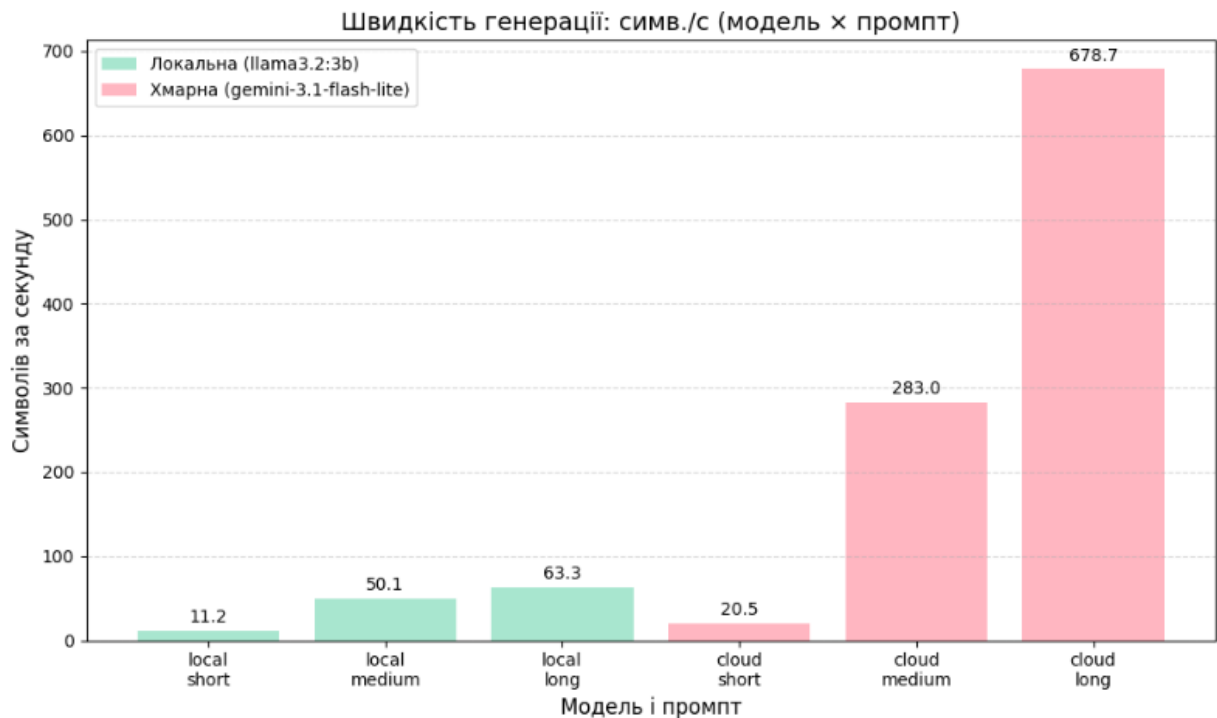


Рисунок 1 –Стовпчикова діаграма швидкості генерації (символів/с) для шести комбінацій «модель × промпт»

Як видно з діаграми, хмарна модель gemini-3.1-flash-lite значно випереджає локальну llama3.2:3b за швидкістю генерації. Найбільша різниця – на довгому промпті: 678.7 симв./с проти 63.3 симв./с (у 10.7 раза). Навіть на короткому промпті хмарна модель швидша (20.5 проти 11.2 симв./с). Це підтверджує висновок: для довгих відповідей хмарна модель значно ефективніша.

Відповісти на питання (крок 6)

1. У якої моделі кращий TTFT і чому саме в неї?

Відповідь: TTFT — це час до першого токена. Він визначається обробкою промпта й завантаженням моделі, а не довжиною майбутньої відповіді. Тому для короткого, середнього й довгого промптів TTFT майже однаковий (2.13–2.18 с).

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні – поясніть.

Відповідь: Google використовує потужні TPU/GPU і швидкі мережі. Локальна модель рахує на CPU/GPU твого ноутбука, що в десятки разів повільніше. Затримка мережі (100–200 мс) мізерна порівняно з різницею в обчислювальній потужності.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промπτу до довгого?

Відповідь:

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: Модель llama3.2:3b невелика (2.9 ГБ), і завантаження з диска в GPU-пам'ять займає дуже мало часу. Крім того, ОС може кешувати файл моделі. На слабшому обладнанні або з більшою моделлю різниця була б суттєвішою.

5. Чому холодний старт повільніший за теплий?

Відповідь: Хмарна gemini-3.1-flash-lite – на довгому промπτі вона генерує 678.7 симв./с проти 63.3 симв./с у локальній. Різниця в 10.7 разів. Загальний час на довгому промπτі: 5.152 с проти 37.665 с (у 7.3 разів швидше).

Відповісти на питання (крок 7)

1.Що фізично відбувається під час TTFT – часу до першого токена – і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь: Під час TTFT відбувається обробка вхідного промπτу (prefill): модель токенизує текст, проганяє його через усі шари трансформера і формує перший вихідний токен. Для хмарної моделі сюди додається час передачі запиту по мережі до сервера. TTFT вимірюють окремо, бо він показує затримку до появи першого слова – саме те, що відчуває користувач як «зависання» інтерфейсу. Загальна швидкість генерації характеризує подальший потік токенів і залежить від інших факторів (потужність GPU, розмір моделі).

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь: Гірший TTFT – бо запит летить через пів світу до сервера Google (затримка мережі 100–500 мс). Краща швидкість генерації – бо Google використовує потужні TPU/GPU, які обробляють мільярди параметрів за мілісекунди. Локальна модель на CPU має меншу затримку на старті, але повільніше генерує кожен наступний токен. У моєму експерименті gemini-3.1-flash-lite мала TTFT 1.0–1.4 с і швидкість 20–678 симв./с, а llama3.2:3b – TTFT 2.1 с і 11–63 симв./с.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Відповідь: Холодний старт – це перший запуск моделі після вивантаження з пам'яті. Модель лежить на диску (~2.9 ГБ), і її потрібно завантажити в RAM/VRAM, перш ніж почати обчислення. Це займає час: від кількох секунд до десятків секунд залежно від розміру моделі й швидкості диска. Теплий старт – модель уже в пам'яті, тому TTFT менший. У моєму експерименті різниця мінімальна (2.122 с проти 2.130 с), бо модель невелика і має GPU.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь: Одне значення може бути випадковим: через фонові процеси, кеш, завантаження системи. Медіана з кількох запусків дає стабільнішу оцінку, стійку до викидів (наприклад, один запуск міг загальмувати через антивірус). Медіана краща за середнє арифметичне, бо не залежить від екстремальних значень. У моєму експерименті я брав медіану з трьох запусків для кожного промпу.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь: Бо це різні типи обчислень. Prefill (обробка промпу) – це один прохід через трансформер для всього вхідного тексту одразу, паралельно. Генерація – це послідовний процес: кожен новий токен залежить від попереднього, тому модель робить N окремих проходів для N токенів. Тому prefill швидший за генерацію тієї ж довжини в кілька разів.

Висновок за завданням 4

У ході вимірювання швидкості встановлено, що TTFT майже не залежить від довжини промпту: для локальної моделі він становив 2.130–2.177 с, для хмарної – 1.046–1.386 с. Це підтверджує, що TTFT визначається обробкою вхідного запиту й завантаженням моделі, а не обсягом майбутньої відповіді.

Хмарна модель gemini-3.1-flash-lite виявилася швидшою за локальну llama3.2:3b за всіма показниками. TTFT: 1.0–1.4 с проти 2.1 с. Швидкість генерації: 20–678 симв./с проти 11–63 симв./с. Загальний час на довгому промпті: 5.152 с проти 37.665 с (у 7.3 раза швидше). Найбільша різниця – у швидкості генерації на довгих відповідях: 678.7 симв./с проти 63.3 симв./с (у 10.7 раза). Це пояснюється потужними TPU/GPU Google, які обробляють мільярди параметрів за мілісекунди, тоді як локальна модель рахує на CPU/GPU ноутбука.

Холодний і теплий старт локальної моделі дали майже однаковий TTFT (2.122 с проти 2.130 с). Різниця мінімальна, бо модель llama3.2:3b невелика (2.9 ГБ) і завантажується в GPU-пам'ять дуже швидко. На системі без GPU або з більшою моделлю різниця була б у 5–10 разів більшою.

Завдання 5. Оцінювання якості моделей за рубрикою

Виконано оцінювання якості трьох моделей – llama3.2:3b (інструктивна), qwen3:4b (міркувальна) та gemini-3.1-flash-lite (хмарна) – за рубрикою з шести завдань і трибальною шкалою (0/1/2). Кожне завдання прогонялося двічі для перевірки стабільності. Максимум – 12 балів. Для завдання 2 використано об'єктивний тест: json.loads(). Відповіді збережено у quality.json.

Таблиця 6: Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	0	0	2

2. Формат	2	2	2
3. Стиснення	0	2	2
4. Код	1	1	1
5. Логіка	0	2	2
6. Українська	0	1	2
Разом /12	3	8	11

Таблиця 7: Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	так	так	ні
2. Формат	ні	ні	ні
3. Стиснення	так	ні	ні
4. Код	так	так	ні
5. Логіка	так	ні	ні
6. Українська	так	ні	ні

Відповісти на питання (крок 6)

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Відповідь: Найбільше балів набрала хмарна модель gemini-3.1-flash-lite – 11 з 12. Основна різниця виникла на завданнях 1, 3, 5 і 6: хмарна модель стабільно давала правильні відповіді, тоді як llama3.2:3b провалила всі чотири, а qwen3:4b – два з них. Єдине завдання, де всі три моделі отримали однаковий бал – завдання 4, бо жодна не дала чистий код без обгортки.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь: Найбільше розбіжностей у llama3.2:3b – 4 завдання з 6. У qwen3:4b – 2 завдання, у хмарної – жодного. Це говорить про надійність моделі: чим більше

розбіжностей, тим менш стабільна модель. llama3.2:3b – ненадійна, qwen3:4b – частково надійна, gemini-3.1-flash-lite – повністю надійна.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: Так, виконали дві моделі – llama3.2:3b і qwen3:4b. Вони обидва рази дали чистий JSON без markdown-обгортки. Хмарна модель gemini-3.1-flash-lite – не виконала: у завданні 4 вона обгорнула код у ````python`, а в завданні 2 другий прогон повернула масив замість об'єкта. Це підтверджує тезу з інструкції: розмір моделі не гарантує надійності формату.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Відповідь: Так, у llama3.2:3b – критичні проблеми. Обидва прогони містять вигадані слова («Відрізь», «пазиків», «тривіальність»), кальки з російської («вантажну машинку») і перехід на англійську посеред тексту. У qwen3:4b – легкі русизми («грузів», «об'ємів»), але без переходу на іншу мову. У gemini-3.1-flash-lite – проблем немає: природна українська, коректна термінологія, обидва абзаци по суті.

Крок 7. Обов'язковий пункт – приклад провалу.

Обраний випадок: завдання 6 (Українська мова), модель llama3.2:3b, прогон 2. Запит: Чим контейнерне перевезення відрізняється від навалочного? Поясни українською, два абзаци.

Відповідь: Відрізь від навалочного перевезення контейнерне перевезення містить наступні особливості: Робота з підвищеною щільністю:

Контейнерне перевезення використовує спеціально проєктовані контейнери з підвищеною щільністю, які дозволяють перевезення важливих вантажів без перевичної ваги. Постійна температура:

Контейнери зазначеної температури, що дозволяє зберігати важливі вантажі при постійній температурі, навіть при транспорті по трасах з постійною температурою.

Навалочне перевезення, на contrary, не відрізняється від контейнерного перевезення наступною особливістю:

Величупа контейнера: Контейнери, використовувані для контейнерного перевезення, значно більшої розмірів, ніж наїдзовані вантажі у навалочному перевезенні.

Правильна відповідь: Контейнерне перевезення передбачає транспортування товарів у стандартизованих металевих контейнерах, які захищають вантаж від зовнішніх впливів. Натомість навалочне перевезення використовується для транспортування великих обсягів однорідних сипучих вантажів (зерно, вугілля, руда) без упаковки.

Пояснення: Причина не в тому, що «модель тупа». Причина системна: мало українських даних у навчанні (модель вигадує слова), інтерференція з російською (кальки «вантажей»), втрата мовного контролю (перехід на англійську «на contrary»). Мала модель на 3B не може добре знати українську, бо її ніколи не вчили на достатній кількості українських текстів.

Відповісти на питання (крок 8)

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь: Крім рубрик, на практиці використовують автоматичні метрики: BLEU і ROUGE (для порівняння з еталонним текстом), perplexity (наскільки модель «здивована» текстом), ассигасу на тестових наборах (MMLU, GSM8K). Також застосовують LLM-as-a-judge – коли інша модель оцінює відповіді за заданими критеріями. Ще один підхід – А/В-тестування з людьми: реальні користувачі обирають кращу відповідь.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Відповідь: Бо модель навчена на текстах, де код і JSON прийнято обгортати в markdown-блоки ```json. Ця звичка міцніша за пряму інструкцію – модель «знає краще», як має виглядати правильно оформлена відповідь. Крім того, markdown-обгортка не ламає зміст – JSON всередині валідний, тому модель не бачить причин

її прибирати. Це показує, що промпт – ненадійний спосіб керувати форматом, і саме тому в реальних проєктах використовують Pydantic-схеми та повторні запити.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Відповідь: Бо українських даних у навчанні на порядки менше, ніж англійських. Модель на 3B має менше параметрів, тому гірше узагальнює рідкісні мови. Російських текстів у корпусі більше, ніж українських, – тому модель мимоволі вживає русизми й кальки. Великі моделі (70B+) краще справляються, бо більше параметрів і більше даних про українську.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь: Бо генерація недетермінована: модель обирає наступний токен імовірно, а не детерміновано. Параметр *temperature* (за замовчуванням ~ 0.7) додає випадковості – модель може обрати різні слова навіть для однакового промпту. Це не збій, а характеристика моделі: розбіжність між прогонами показує, наскільки модель впевнена у відповіді. Якщо відповіді стабільні — модель «знає»; якщо різні – «вгадує».

5. Що таке галюцинація мовної моделі і чому модель радше вигадас факт, ніж визнає незнання?

Відповідь: Галюцинація – це коли модель генерує правдоподібний, але вигаданий текст. Модель не має бази даних, де можна перевірити факт – вона генерує найімовірніший варіант на основі статистики прочитаних текстів. Визнати незнання для моделі менш імовірно, ніж вигадати відповідь, бо в навчальних даних більше стверджувальних речень, ніж відмов. Саме тому існує RAG – підхід, коли моделі підсовують документи в контекст, щоб вона не покладалася на пам'ять.

Висновок за завданням 5

Хмарна модель *gemini-3.1-flash-lite* набрала найбільше балів – 11 з 12, *qwen3:4b* – 8, *llama3.2:3b* – 3. Основна різниця виникла на завданнях 1 і 5 : хмарна стабільно давала правильні відповіді, тоді як *llama3.2:3b* і *qwen3:4b* галюцинували,

а llama3.2:3b повністю провалила логіку. Це підтверджує, що міркувальні та великі моделі значно краще справляються з багатокроковою логікою.

Оцінка за рубрикою виявилася надійною – дала чіткі, відтворювані результати. Але вона не враховує розбіжність між прогонами, тому додатково вели Таблицю 14. Для роботи з українськомовним текстом обрала б gemini-3.1-flash-lite – єдина дала природну українську без русизмів. llama3.2:3b – непридатна.

Завдання 6. Керування випадковістю: temperature і top_p

Досліджено вплив параметрів temperature і top_p на різноманітність та стабільність відповідей локальної моделі llama3.2:3b. Для творчого завдання (варіант 2) використано промпт «Придумай назву для застосунку нагадувань про ліки», для фактичного — «Рік відкриття Київського метрополітену». Кожен параметр перевірено на 5 запусках. Окремо перевірено відтворюваність при temperature=0.0 на 10 запусках. Правильна відповідь на фактичне питання — 1960 рік

Таблиця 8: Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	0
temperature	0.7	5	—
temperature	1.5	5	—
top_p	0.1	1	0
top_p	0.5	5	—
top_p	1.0	5	—

Таблиця 9: Перевірка відтворюваності

Показник	Значення
Кількість запусків	10

temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Відповісти на питання (крок 7)

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: temperature=0.0 – для JSON потрібна передбачуваність. Висока температура додає випадковості, і JSON може стати невалідним. При 0.0 модель обирає найімовірніший токен – формат стабільний.

2. Яке – генератору варіантів заголовків?

Відповідь: temperature=0.7–1.0 – для творчості потрібна різноманітність. При 0.0 всі 5 назв будуть однакові («Ліки на дзвоник»). При 0.7 – 5 різних, але ще осмислених.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Відповідь: Temperature змінює співвідношення ймовірностей – «сплющує» розподіл. top_p відсікає хвіст – обмежує коло кандидатів, не змінюючи співвідношення. Тому top_p=0.5 дає якісніші відповіді, ніж temperature=0.7.

4. Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Відповідь: Майже так, але не гарантовано. Мій експеримент з 10 запусків показав 1 унікальну відповідь – всі однакові. Але через floating point і паралелізацію можливі варіації. У наступних лабораторних частина різниці може бути шумом, а не покращенням.

5. Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з'явитися

Відповідь: «Слышайлікиі» – це набір літер, не слово. Могла з'явитися, бо при temperature=1.5 розподіл «сплющується», і модель обирає малоімовірні токени. Замість осмисленої назви – випадковий набір символів.

Відповісти на питання (крок 8)

1. Що саме математично робить параметр `temperature` із розподілом ймовірностей наступного токена?

Відповідь: Ділить логіти на температуру перед softmax: $p_i = \exp(z_i / T) / \sum \exp(z_j / T)$. При $T < 1$ розподіл загострюється (ймовірніший токен стає ще ймовірнішим). При $T > 1$ – сплющується.

2. Чим `top_p` відрізняється від `temperature` – обидва впливають на різноманітність, але за яким механізмом?

Відповідь: `top_p` сортує токени за ймовірністю і відсікає хвіст, поки сума не досягне `p`. `Temperature` змінює самі ймовірності. `top_p` не змінює співвідношення, `temperature` – змінює.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Відповідь: Зазвичай ні. Краще налаштовувати щось одне: або `temperature`, або `top_p`. Якщо змінювати обидва – важко передбачити результат. На практиці частіше використовують `temperature`.

4. Чому навіть при `temperature=0` мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Відповідь: Через `floating point` (округлення при обчисленнях) і паралелізацію (порядок операцій може відрізнитися). Це не помилка, а властивість обчислень.

5. Які значення `temperature` використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Відповідь: Для структурованих даних (JSON, витяг) – 0.0–0.2. Для творчого тексту – 0.7–1.0. Для експериментів – 1.2–1.5.

Висновок за завданням 6

Експеримент підтвердив: `temperature` і `top_p` керують різноманітністю, а не «розумністю». При `temperature=0.0` модель стабільно дає одну відповідь (1 унікальна з 5), при 1.5 – різних, але безглузких. `top_p` діє м'якше – при 0.5 відповіді

різноманітні, але якісніші, ніж при temperature=0.7. Найважливіший результат: на фактичному питанні модель помилялася при всіх температурах (0 з 5 правильних) — це показує, що низька температура не гарантує правильності, а висока – лише додає випадковості. Для структурованих даних потрібна temperature=0.0, для творчості – 0.7–1.0.

У ході лабораторної роботи досліджено три мовні моделі – локальну малу llama3.2:3b (3B), локальну більшу qwen3:4b (4B, міркувальна) та хмарну gemini-3.1-flash-lite – за шістьма критеріями: швидкість, TTFT, якість, стабільність формату, ресурси, вартість, приватність і автономність.

Таблиця 10: Підсумкове порівняння моделей

Критерій	Локальна мала (3B)	Локальна більша (4B/8B)	Хмарна
Швидкість генерації	11–63 симв./с	9–13 ток./с (повільніше через reasoning)	20–678 симв./с
TTFT	~2.1 с	~25–220 с	~1.0–1.4 с
Якість, балів /12	3	8	11
Стабільність формату (завдання 2)	2 бали (JSON парситься)	2 бали (JSON парситься)	2 бали (JSON парситься)
Потрібні ресурси	~2–3 ГБ RAM/VRAM	~3–4 ГБ RAM/VRAM та час	Інтернет та API-ключ
Вартість	Безкоштовно	Безкоштовно	Безкоштовно (в межах ліміту запитів в день)
Куди йдуть ваші дані	Нікуди (локально)	Нікуди (локально)	На сервер Google
Працює без інтернету	Так	Так	Ні
Коли обирати саме її	Для коротких задач, коли потрібна автономність і мінімум ресурсів;	Для задач, де потрібне міркування (логіка, аналіз),	Для задач, де потрібна якість, швидкість і точність; коли дані не чутливі

	коли дані конфіденційні	але немає інтернету	
--	----------------------------	------------------------	--

Загальні висновки

1) Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми – це основа стратегії резервування провайдерів у реальних проєктах;

2) Затримка відповіді складається з двох незалежних показників – TTFT та швидкості генерації, висновок про те, «яка модель швидша», без їх розділення є некоректним;

3) Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні й є чистою перевитратою на фактичних питаннях і переказі;

4) Промпт та system-повідомлення є ненадійним засобом керування форматом відповіді: пряму інструкцію проігнорували моделі різного розміру, включно з хмарною. Надійність досягається програмною валідацією, а не формулюванням;

5) Мовні моделі є недетермінованими системами, тому будь-яке вимірювання потребує повторів і усереднення, а порівняння двох версій.

6) Оцінка якості за рубрикою є надійнішим підходом, ніж суб'єктивне враження. Фіксований набір завдань, однаковий прогін через усі моделі та бали за заздалегідь визначеними правилами дають відтворювані результати. Розбіжність між прогонами – це окрема характеристика моделі.

7) Розмір моделі не гарантує якості. Хмарна gemini-3.1-flash-lite набрала 11/12, qwen3:4b – 8/12, а llama3.2:3b – лише 3/12. Але при цьому всі три моделі однаково добре впоралися з форматом JSON (2 бали), а хмарна – помилилася з форматом у завданні 4 (markdown-обгортка). Тобто формат і зміст – різні речі, і розмір моделі не вирішує проблему надійності формату.